

Tech & Teach

Digital Chip Design Program

Design RTL

names:

Shaher Alharbi

Suliman w Alrumaikani

Abdullah AlJumah

Tech & Teach

Task :06

2026\07\09

Task topic:

Version 1.0

Task: 1.1

1.1.1.1

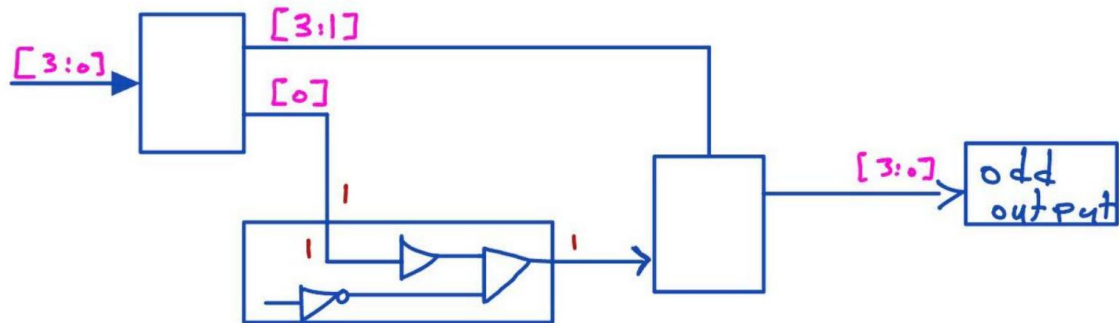
```
1 timescale 1ns / 1ps
2
3 module toOdd (
4     input logic [3:0] n,
5     output logic [3:0] oddN
6 );
7
8     always_comb begin
9         if (n[0] == 1'b0) begin
10             oddN = n + 4'b0001;
11         end else begin
12             oddN = n;
13         end
14     end
15 endmodule
```



```
1 timescale 1ns / 1ps
2
3 module TB_toOdd;
4     logic [3:0] n;
5     logic [3:0] oddN;
6     toOdd uut (
7         .n(n),
8         .oddN(oddN)
9     );
10    initial begin
11        for (int i = 0; i < 16; i++) begin
12            n = i;
13            #10;
14        end
15    end
16 endmodule
```

Name	Value	999,995 ps	999,996 ps	999,997 ps	999,998 ps	999,999 ps	1,000,000 ps
▼ n[3:0]	f						
▼ [3]	1						
▼ [2]	1						
▼ [1]	1						
▼ [0]	1						
▼ oddN[3:0]	f						
▼ [3]	1						
▼ [2]	1						
▼ [1]	1						
▼ [0]	1						

1.1.1.2



1.1.1.3

```

1  `timescale 1ns / 1ps
2  module toOdd (
3      input  logic [3:0] n,
4      output logic [3:0] oddN
5  );
6      logic [3:0] n_plus_one;
7      logic sel;
8      assign n_plus_one = n + 4'b0001;
9      assign sel = n[0];
10     assign oddN = (sel == 1'b0) ? n_plus_one : n;
11 endmodule

```

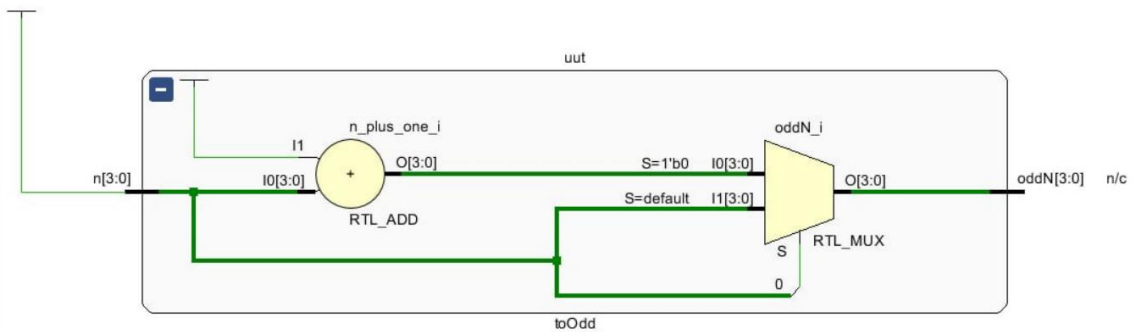
[3:0]

Tech & Teach

```

1  `timescale 1ns / 1ps
2
3  module TB_toOdd;
4      logic [3:0] n;
5      logic [3:0] oddN;
6      toOdd uut (
7          .n(n),
8          .oddN(oddN)
9      );
10     initial begin
11     for (int i = 0; i < 16; i++) begin
12         n = i;
13         #10;
14     end
15     end
16 endmodule

```

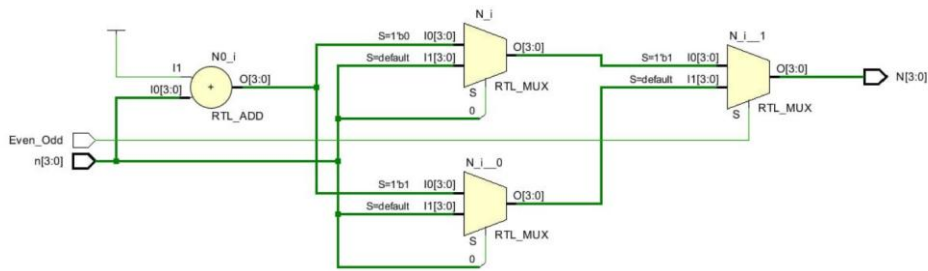


1.1.1.4

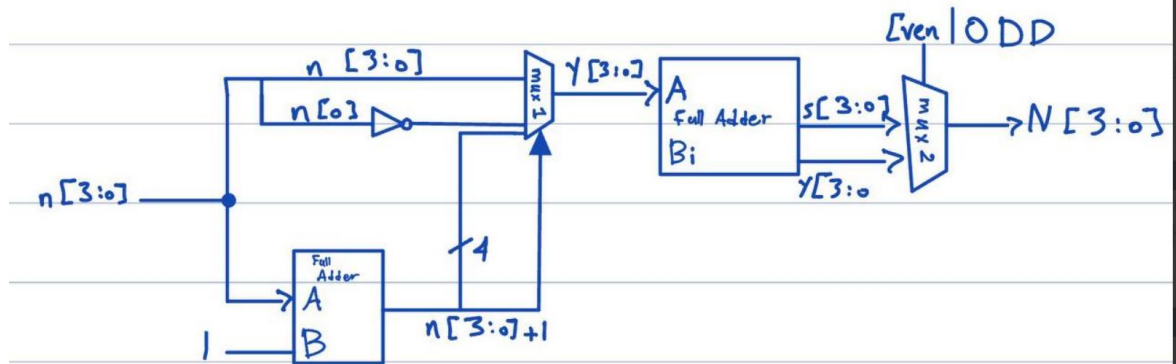
```

1  `timescale 1ns / 1ps
2  module toOdd (
3      input  logic [3:0] n,
4      output logic [3:0] oddN
5  );
6      logic [3:0] n_plus_one;
7      logic sel;
8      assign n_plus_one = n + 4'b0001;
9      assign sel = n[0];
10     assign oddN = (sel == 1'b0) ? n_plus_one : n;
11 endmodule

```

1.1.2.2

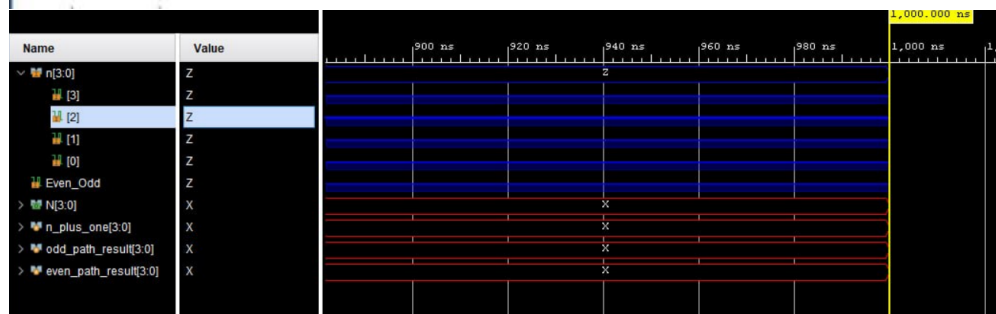


1.1.2.3

```

1 timescale 1ns / 1ps
2
3 module toEvenOdd (
4     input logic [3:0] n,
5     input logic Even_Odd,
6     output logic [3:0] N
7 );
8 logic [3:0] n_plus_one;
9 logic [3:0] odd_path_result;
10 logic [3:0] even_path_result;
11 assign n_plus_one = n + 4'b0001;
12 assign odd_path_result = (n[0] == 1'b0) ? n_plus_one : n;
13 assign even_path_result = (n[0] == 1'b1) ? n_plus_one : n;
14 assign N = (Even_Odd == 1'b1) ? odd_path_result : even_path_result;
15 endmodule

```

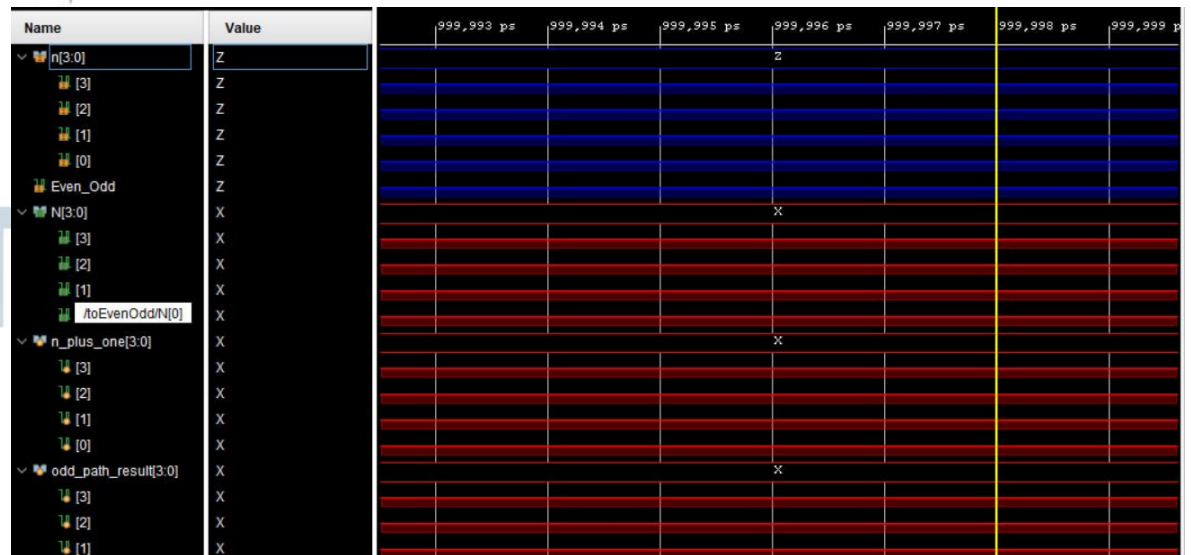


1.1.2.4

```

1  `timescale 1ns / 1ps
2
3  module TB-toEven:
4  logic [3:0] n;
5  logic      Even_Odd;
6  logic [3:0] N;
7  toEvenOdd uut (
8  .n(n),
9  .Even_Odd(Even_Odd),
10 .N(N)
11 );
12     initial begin
13 Even_Odd = 1'b0;
14 for (int i = 0; i < 16; i++) begin
15 n = i;
16 #10;
17 end
18 #20;
19 Even_Odd = 1'b1;
20 for (int i = 0; i < 16; i++) begin
21 n = i;
22 #10;
23 end
24 end
25 endmodule

```

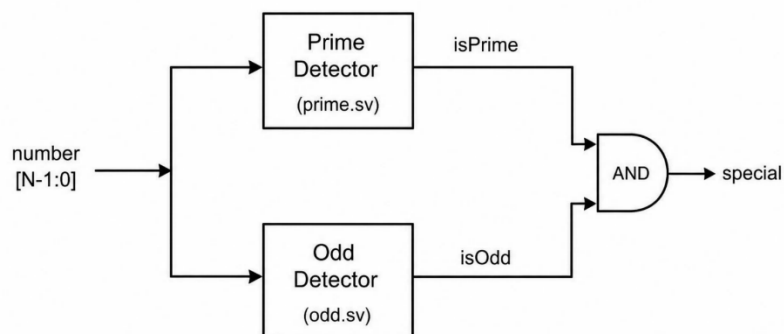


Task: 1.2

1.2.1.1

```
1 module top #(parameter N = 3) (  
2   input logic [N-1:0] number,  
3   output logic special  
4 );  
5 logic isPrime, isOdd;  
6 prime #(N) p1(  
7   .number(number),  
8   .isPrime(isPrime)  
9 );  
10 odd #(N) o1(  
11   .number(number),  
12   .isOdd(isOdd)  
13 );  
14 and (special, isPrime, isOdd);  
15  
16 endmodule
```

1.2.1.2



1.2.1.3


```

1 module odd #(parameter N = 3) (
2
3     input  logic [N-1:0] number,
4     output logic isOdd
5
6 );
7
8 assign isOdd = number[0];
9
10 endmodule

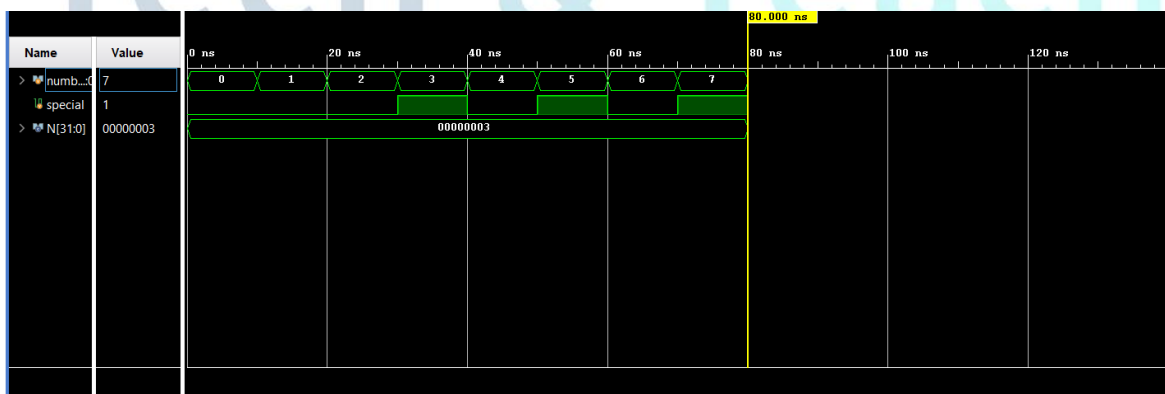
```

```

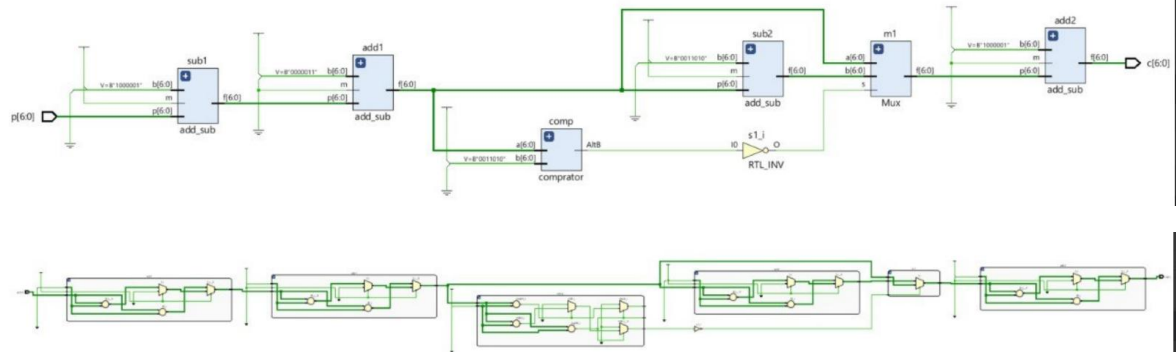
1 module top_tb;
2
3     parameter N = 3;
4     logic [N-1:0] number;
5     logic special;
6     top #(N) DUT (
7         .number(number),
8         .special(special)
9     );
10    initial begin
11        number = 3'd0; #10;
12        number = 3'd1; #10;
13        number = 3'd2; #10;
14        number = 3'd3; #10;
15        number = 3'd4; #10;
16        number = 3'd5; #10;
17        number = 3'd6; #10;
18        number = 3'd7; #10;
19
20    $finish;

```

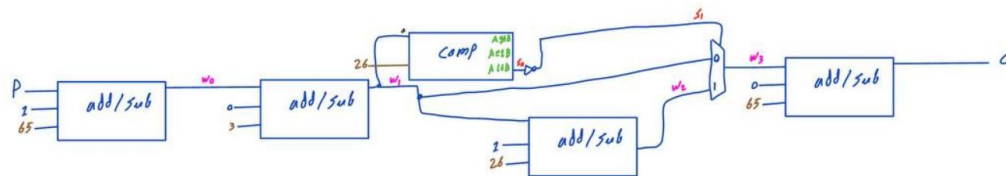
1.1.2.4



1.3.1.1



1.3.1.2



1.3.1.3

Tech & Teach

```

1 module top(
2   input logic [6:0]p ,
3   output logic [6:0]c);
4   logic [6:0] w0,w1,w2,w3;
5   logic s0 , s1;
6   //convert from 65-90 to 0-25
7   add_sub sub1(
8     .p(p),
9     .m(1),
10    .b(65),
11    .f(w0)
12  );
13  // add 3 for encryption
14  add_sub add1(
15    .p(w0),
16    .m(0),
17    .b(3),
18    .f(w1)
19  );
20  //compare the ascii with the out-range number 26
21  comparator comp(
22    .a(w1),
23    .b(26),
24    .AltB(s0),
25    .AeqB(),
26    .AgtB()
27  );
28  not (s1,s0);

```



Tech & Teach

```

9
10 // if the number after addetion >= 26 we will sub 26
11 add_sub sub2(
12     .p(w1),
13     .m(1),
14     .b(26),
15     .f(w2)
16 );
17
18 //dpending to the s1 we will pass rather w1 or w2
19 Mux m1(
20     .s(s1),
21     .a(w1),
22     .b(w2),
23     .f(w3)
24 );
25
26 // we add 65 to get it agin to the ascii code number
27 add_sub add2(
28     .p(w3),
29     .m(0),
30     .b(65),
31     .f(c)
32 );
33 endmodule

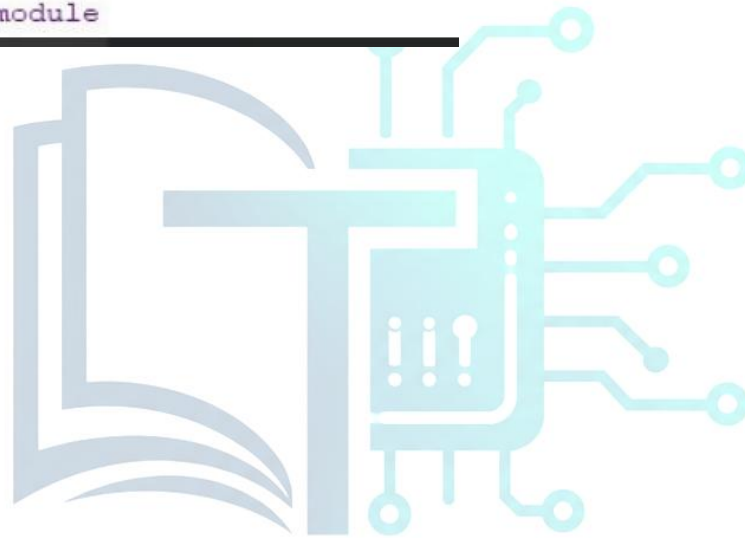
```

```

1 module Mux(
2     input logic s,
3     input logic [6:0] a,b,
4     output logic [6:0]f
5 );
6
7     assign f = (s)? b : a;
8 endmodule
9

```

```
1 module add_sub(  
2     input logic [6:0]p,b,  
3     input logic m,  
4     output logic [6:0]f  
5 );  
6     always_comb begin  
7         if(m==1)  
8             f = p - b;  
9         else if (m==0)  
10            f = p + b;  
11        else  
12            f = 6'bxx;  
13        end  
14    endmodule
```



Tech & Teach

```

1  module comparator(
2      input logic [6:0] a,b,
3      output logic AgtB , AeqB , AltB
4          );
5
6      always_comb begin
7          if(a > b)begin
8              AgtB = 1;
9              AeqB = 0;
10             AltB = 0;
11         end
12         else if(a == b)begin
13             AgtB = 0;
14             AeqB = 1;
15             AltB = 0;
16         end
17         else if(a < b)begin
18             AgtB = 0;
19             AeqB = 0;
20             AltB = 1;
21         end
22         else begin
23             AgtB = 1'bX;
24             AeqB = 1'bX;
25             AltB = 1'bX;
26         end
27     end
28 endmodule
29

```



reach

```

1 module task3 (
2   input a3,input b3,input a2,input b2,input a1,input b1,input a0,input b0,
3   output AeqB,output AltB,output AgtB);
4   logic w0 , w1 , w2 ,w3;
5   logic nb3 , nb2 , nb1 , nb0;
6   logic w4 , w5 , w6 , w7;
7   //the first layer
8   xnor (w0,a3,b3);
9   xnor (w1,a2,b2);
10  xnor (w2,a1,b1);
11  xnor (w3,a0,b0);
12  //the second layer upper part(output from past input here)
13  and (AeqB,w0,w1,w2,w3);
14  //lower part in second layer
15  not (nb3,b3);
16  not (nb2,b2);
17  not (nb1,b1);
18  not (nb0,b0);
19
20  and (w4,nb3,a3);
21  and (w5,w0,nb2,a2);
22  and (w6,w0,w1,nb1,a1);
23  and (w7,w0,w1,w2,nb0,a0);
24
25  or (AgtB,w4,w5,w6,w7);
26  //last layer
27  nor (AltB,AeqB,AgtB);
28 endmodule

```

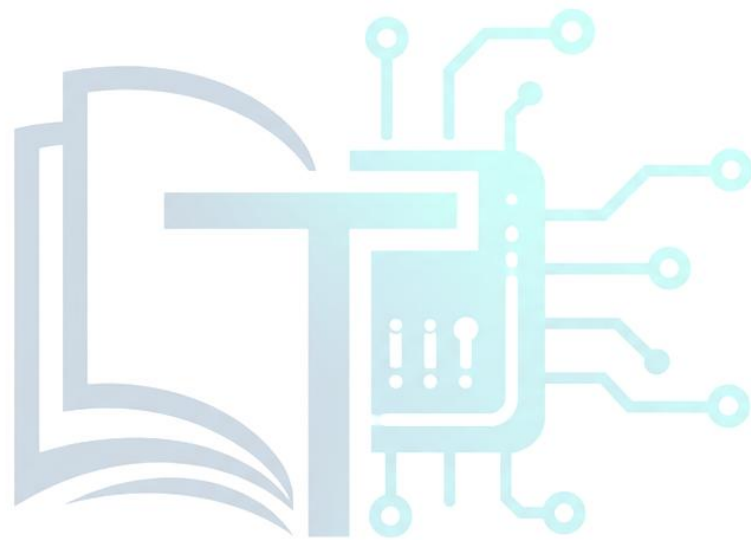
1.3.1.4

```

1   `timescale 1ns / 1ps
2
3   module testB( );
4     logic [6:0]p;
5     logic [6:0]c;
6
7     top dut(
8       .p(p),
9       .c(c)
10    );
11
12    initial begin
13      p = 66;
14
15      #30 p = 90;
16
17      #30 p = 88;
18
19      #30 p = 70;
20    end
21  endmodule

```





Tech & Teach

Work distribution

Section	shaher	Suliman	Abdullah
Report	✓		
Task 1.1			✓
Task 1.2	✓		
Task 1.3		✓	

Tech & Teach